sunghwan@qa: ~/portfolio

```
sunghwan@qa:~/portfolio$ ./run_test.sh --list
```

TEST SUITE · CONTENTS

목차

각 TC는 테스트 방법(Part 1) 바로 뒤에
샘플 제품으로 재현한 결과(Part 2)가 이어진다

LEGEND

Part 1 실무 경력 · 방법

Part 2 샘플 제품 Dice Duel로 재현

```
$ ./run_test.sh --list # 12 pages

[----------] Intro · 나는 누구인가
[ 01 ] 목차 contents
[ 02 ] TC-001 · 소개 profile Part 1

[----------] TC-002 · 테스트 설계와 수행
[ 03 ] TC-002 · 테스트 설계와 수행 Sanity / Patch Part 1
  [ 04 ] Dice Duel QA 재현용 샘플 제품 개요 Part 2
  [ 05 ] TC-002 · 설계와 준비 Step 01–04 Part 2
  [ 06 ] TC-002 · 검증과 결함 리포트 Step 05–06 Part 2

[----------] TC-003 · 테스트 자동화
[ 07 ] TC-003 · 테스트 자동화 Web UI + Server Part 1
  [ 08 ] TC-003 · 테스트 자동화 Cucumber · Jenkins · log 모니터링 Part 2

[----------] TC-004 · 성능 테스트
[ 09 ] TC-004 · 성능 테스트 Instance scaling Part 1
  [ 10 ] TC-004 · 성능 테스트 인스턴스 1 → 3개 측정 Part 2

[----------] Career · 경력과 도구
[ 11 ] 경력 마일스톤 2012 – 2026 Part 1
[ 12 ] Appendix 전체 도구 목록 Part 1
```

repo: github.com/Juanito-kim/dice-duel-qa · Part 2 산출물 전체

report: juanito-kim.github.io/dice-duel-qa · Cucumber 리포트

sunghwan@qa: ~/portfolio

```
sunghwan@qa:~/portfolio$ ./init_database.sh --profile kim-seonghwan
```

TEST CASE · TC-001

QA Engineer 김성환 소개

Automotive Telematics부터 반도체 Fab 수율 관련 솔루션까지,
12년 10개월 동안 SW 검증을 해온 QA Engineer입니다.

RESULT
6/6 PASSED
[#######] 100%

```
sql> SELECT * FROM profile;

objective:      제품 품질을 믿고 맡길 수 있는 QA인지 검증
precondition:   - QA 경력 12년 10개월
                - 도메인: Automotive Telematics · 반도체 Fab 수율 관련 솔루션
environment:    [Kubernetes, Selenium, Windmill, Grafana, Prometheus, Jira]
labels:         test-design | performance-testing | test-automation | regression-testing | exploratory-testing
                defect-tracking | ai-assisted-testing | domain-automotive | domain-semiconductor
```

```
[==========] Running 6 steps from TC-001

[ PASSED ] 01 처음 보는 도메인을 맡긴다
                └── Automotive에서 Semiconductor로 도메인을 옮겨가며 검증을 수행한다

[ PASSED ] 02 명세(Requirements)를 받는다 #requirements #test-design
                └── 명세 기반으로 테스트 케이스와 유저 시나리오를 도출한다

[ PASSED ] 03 테스트를 수행한다 #jira
                └── 결함은 Jira로 리포팅·추적하고, 결함이 없음도 근거로 확인한다

[ PASSED ] 04 반복되는 Web UI 회귀 테스트를 맡긴다 #regression-test
                └── Selenium·Cucumber + Jenkins로 자동화하고, AI로 스크립트 작성 속도를 높인다

[ PASSED ] 05 명세에 없는 시나리오를 맡긴다 #exploratory-test
                └── 탐색적 테스트로 문서화되지 않은 결함을 찾아낸다

[ PASSED ] 06 시스템의 성능 검증을 맡긴다 #stress-test #k8s-pod-scaling
                └── 최적 HW 사양 결정을 위한 서버 스펙을 측정한다

[==========] 6 steps ran.
```

---

actual_result: 12년 10개월간 3개 회사에서 반복 검증 완료

status: **PASS**

```
sunghwan@qa:~/portfolio$
```

```
sunghwan@qa:~/portfolio$ ./run_test.sh --case TC-002
```

TEST CASE · TC-002

테스트 설계와 수행

Test Design & Execution · Sanity / Patch
빌드·패치마다, 설계부터 시작해 바뀐 것까지 검증한다

RESULT
6/6 PASSED
[######] 100%

```
$ cat tc-002.yaml
```

company: 네오디지탈테크 · 비스텔 · 시높시스코리아
objective: 새 빌드·패치가 설치 명세대로 동작하고, 패치 아이템이 의도대로 반영됐는지 검증
precondition:
- 새 빌드 또는 패치 릴리스
- Installation spec, 패치 아이템 목록

environment: [Windows, Linux, Kubernetes (helm chart), Jira, Zephyr]
labels: test-case-design, test-environment-setup, installation-test, sanity-test, patch-verification, manual-testing

[==========] Running 6 steps from TC-002

[PASSED] 01 **Customer requirements를 검토한다** #requirements KEY
└ 고객 요구사항과 Installation spec, 패치 아이템 목록을 분석한다

[PASSED] 02 **테스트 케이스 및 유저 시나리오를 작성한다** #test-design #user-scenario KEY
└ 테스트 수행 전 필수 작업. 요구사항과 패치 아이템을 기준으로 테스트 케이스와 유저 시나리오를 작성하고 Zephyr (for Jira)로 관리한다

[PASSED] 03 **환경을 구성하고 빌드·패치를 설치한다** #installation-test
└ Installation spec에 맞춰 환경을 구성하고, 설치 후 서비스 상태와 신뢰성을 검증한다

[PASSED] 04 **Sanity test를 수행한다** #sanity
└ 핵심 기능이 정상 동작하는지 빠르게 확인한다

[PASSED] 05 **패치 아이템을 검증한다** #patch #manual-test
└ 패치 아이템별 기능을 검증하고, 결함은 Jira로 리포팅한다

[PASSED] 06 **회귀 시나리오에 반영한다** #bdd
└ 패치로 바뀐 동작은 기존 BDD 시나리오를 업데이트한다 → TC-003

[==========] 6 steps ran.

---

actual_result: 빌드·패치마다 설계 → 설치 → Sanity → 패치 검증 → 회귀 반영을 반복
next: → Part 2 · Dice Duel v1.0 · TC 45건 수행, 결함 4건·명세 이슈 2건 리포트
status: **PASS**

sunghwan@qa: ~/dice-duel-qa

```
sunghwan@qa:~/dice-duel-qa$ git clone https://github.com/Juanito-kim/dice-duel-qa
```

PART 2 · SAMPLE PROJECT

Dice Duel QA

샘플 제품을 직접 만들고, Part 1의 테스트를 그대로 재현했다
제품은 최소한으로, 테스트 산출물은 최대한으로

RESULT

3/3 REPRODUCED

TC-002 · TC-003 · TC-004

```
$ cat project.yaml
```

product: Dice Duel v1.0 · 플레이어와 컴퓨터가 주사위를 던져 합이 큰 쪽이 이기는 미니게임 (먼저 3승)

architecture: 실제 사용자(브라우저) · 가상 사용자(봇, Generator) → nginx → app × N (FastAPI) → PostgreSQL, log → log 모니터 (ERROR 필터링)

environment: [Python (FastAPI), PostgreSQL, nginx, Docker Compose, Java, Cucumber, Selenium, Jenkins, GitHub Actions, Claude Code]

repo: github.com/Juanito-kim/dice-duel-qa

report: juanito-kim.github.io/dice-duel-qa · Cucumber 리포트

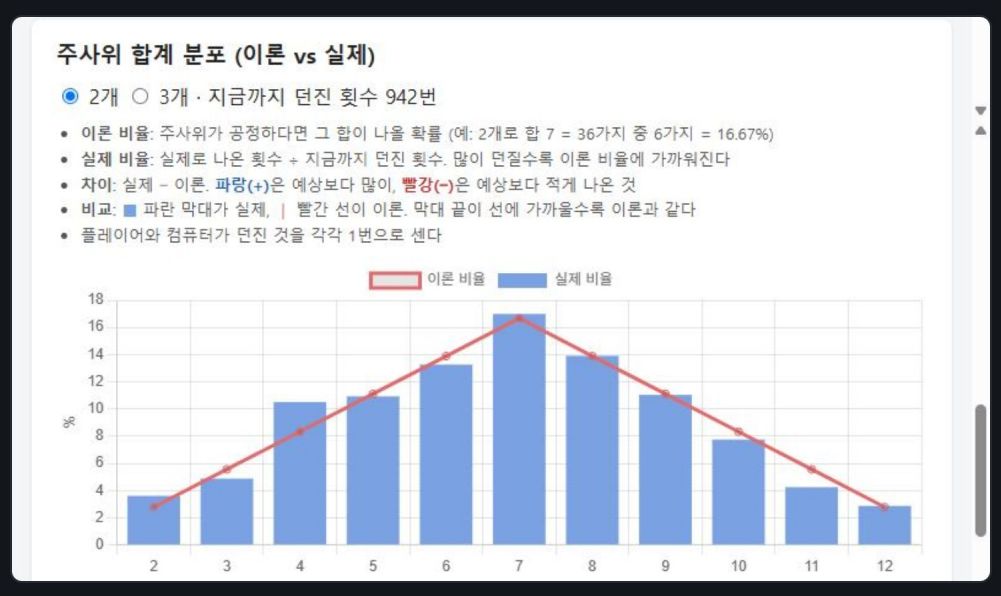
[UI] 게임 화면 #web-ui



먼저 3승하면 매치가 끝나고 던지기 버튼이 비활성화된다 (R-03, R-08)

LINK ↗ app/static/game.html

[UI] 통계 화면 #statistics



주사위 합계 분포를 이론 비율과 비교한다. Chart 옆에 같은 값을 표로도 둬서 자동화로 검증

LINK ↗ app/static/stats.html

```
$ cat results.summary # 수행 결과
```

TC-002 · 테스트 설계와 수행	TC 45건 작성·수행: PASS 37 · FAIL 6 · BLOCKED 2
└── 결함 · 명세 이슈	BUG 4건 (Major 3 · Minor 1) · 명세 이슈 SPEC 2건
TC-003 · 테스트 자동화	Cucumber 19 시나리오: PASS 15 · 알려진 결함 FAIL 4 · 새 실패 0
└── Server log 모니터링	의도적으로 주입한 오류 17건 = 검출 17건 (ERROR 12 · WARNING 5)
TC-004 · 성능 테스트	인스턴스 1→3개: 188→534 요청/초 (×2.84), 오류 0

* BUG-001~003은 결함 리포트를 보여주기 위해 미리 심은 결함이며 위치와 의도를 planted-bugs.md에 공개했다. BUG-004와 SPEC 2건은 테스트하며 실제로 발견했다.

---

actual_result: Part 1의 TC-002~004를 샘플 제품에서 모두 재현하고, 산출물과 실행 기록을 GitHub에 공개

start_here: README.md · "QA 과정 한눈에 보기"

status: PASS

sunghwan@qa: ~/dice-duel-qa

```
sunghwan@qa:~/dice-duel-qa$ ./run_test.sh --case TC-002 --target dice-duel-v1.0 --steps 01-04
```

PART 2 · TC-002 · STEP 01-04

테스트 설계와 준비

명세를 검토해 테스트를 설계하고
설치와 Sanity로 테스트할 수 있는 빌드인지 확인한다

RESULT
11/11 PASSED
Sanity · 클린 설치 19초

```
$ diff part1/TC-002 part2/dice-duel # 실무와 다른 점
```

PART 1 · 실무		PART 2 · DICE DUEL (Sample project)
고객 요구사항	→	요구사항 명세 (규칙 ID R-01~R-09 등)
Installation spec	→	03-install.md (Docker Compose 설치 절차)
패치 아이템	→	v1.0 전체 기능 (첫 릴리스로 가정)
Zephyr (for Jira)	→	Markdown TC 표 45건 (TC ID · 규칙 ID · 사전 조건 · 기대 결과 · 자동화 여부) LINK ↗

[01] 요구사항 검토 #requirements

KEY

01. 요구사항 명세 — Dice Duel v1.0

> Part 1 · TC-002 · 01 요구사항 검토 작성: 김성환 · 확정일: 2026-10-26

플레이어와 컴퓨터가 주사위를 던져 합이 큰 쪽이 이기는 미니게임이다. 매치 결과를 기록하고 통계를 보여준다. 이 문서의 규칙 ID(R-xx)는 테스트 케이스, Gherkin 태그(`@R-02`), 결함 리포트에서 그대로 참조한다.

1. 게임 규칙 (Rule)

ID	규칙	검증 포인트
R-01	라운드마다 플레이어와 컴퓨터가 같은 개수의 주사위를 던져, 눈의 합이 큰 쪽이 라운드 승리	합 비교, 양쪽 주사위 개수 동일
R-02	합이 같으면 모든 주사위 눈이 같은 쪽이 승리한다. 2개 매치는 **더블**(예: [3,3]). 3개 매치는 **트리플**(예: [2,2,2])만 해당한다. 양쪽 모두 해당하거나 모두 해당하지 않으면 **무승부**	3개 매치의 [4,4,1]은 트리플이 아님
R-03	먼저 3라운드를 이긴 쪽이 매치 승리. 3승 즉시 매치가 끝난다. 무승부 라운드는 승수에 포함하지 않는다	3승 직후 종료, 무승부 라운드 무시
R-04	10라운드가 끝나도 3승이 없으면 승수가 많은 쪽이 승리, 같으면 매치 무승부	10라운드 종료 판정 (예: 2승 2패 6무 → 무승부)
R-05	주사위 개수는 2개 또는 3개. 매치 시작 전에만 고를 수 있다	그 외 값 거부, 매치 중 변경 불가
R-06	닉네임은 2~10자, 한글·영문·숫자만 허용. 앞뒤 공백도 허용하지 않는다. 그 외는 매치 시작을 거부하고 **ERROR log**를 남긴다	경계값 1·2·10·11자, 특수문자, 공백
R-07	주사위는 **서버**에서 굴린다. 매치마다 seed를 저장하고, 같은 seed면 같은 결과가 나온다. 특정 눈이 필요한 테스트는 **테스트 모드 전용** API로 다음 라운드의 눈을 지정한다	seed 재현성, 테스트 모드가 아니면 지정 API 거부
R-08	매치가 끝나면 던지기 불가. 끝난 매치에 던지기 요청이 오면 거부하고 **ERROR log**를 남긴다	화면 버튼 비활성화 + 서버 거부
R-09	던지기 요청에는 라운드 번호가 들어간다. 같은 라운드에 요청이 중복되면 **한 번만 처리**하고 나머지는 거부하며 **WARN log**를 남긴다	동시 요청 2건 → 1건 성공, 1건 거부

규칙마다 ID(R-01~R-09 등)를 붙여 TC·Gherkin 태그·결함과 추적한다. 모호한 점 8건(Q-1~Q-8)은 결정으로 기록

LINK ↗ docs/01-requirements.md

[02] 유저 시나리오 작성 #user-scenario

02. 유저 시나리오 — Dice Duel v1.0

> Part 1 · TC-002 · 02 유저 시나리오 작성 상태: **확정 (2026-10-27)** · 기준 문서: 01-requirements.md · 테스트 케이스: 02-test-cases.md

TC가 **규칙 하나**를 검증한다면, 유저 시나리오는 **사용자의 목표 하나**를 처음부터 끝까지 따라간다. 규칙 단위로는 보이지 않던 빈틈(중간에 나가면? 잘못 입력하면?)을 찾는 것이 목적이다.

요약

ID	사용자 목표	거치는 규칙	관련 TC	시나리오에서 찾은 것
US-01	처음 온 사용자가 매치 한 판을 끝까지 한다	U-01, R-06, R-01~R-03, R-08	A02, B01~B06, C01	—
US-02	닉네임을 잘못 넣었다가 고쳐서 시작한다	R-06, U-01	A01, A03, A05, A08	화면 안내("2~10자")와 실제 동작 비교
US-03	매치 도중에 페이지를 닫고 떠난다	(없음)	G01, F13	**처리 규칙 없음 → SPEC-001**
US-04	통계 화면에서 지난주 내 기록을 찾아본다	S-01, U-02, S-04	F01~F05, F14, F15	**기간을 거꾸로 넣으면? → SPEC-002**
US-05	주사위 3개로 길게 겨루다 10라운드까지 간다	R-05, R-02, R-04	A10, B07, B08, C03, C04	매치 중에도 주사위 개수 선택이 바뀌어 보임

규칙 단위 TC로는 안 보이는 질문(게임 중간에 브라우저를 떠나면? 기간을 거꾸로 넣으면?)을 찾아 명세 이슈 2건으로 연결

LINK ↗ docs/02-user-scenarios.md

[02] 테스트 케이스 작성 #test-design

KEY

02. 테스트 케이스 — Dice Duel v1.0

> Part 1 · TC-002 · 02 테스트 케이스 작성 상태: **확정 (2026-10-27)** · 대상: Dice Duel v1.0 기준 문서: 01-requirements.md · 사용자 흐름: 02-user-scenarios.md

대표 TC (기능별 1건)

전체 TC는 모두 수행하지만, 포트폴리오·README에는 기능별로 **대표 1건**만 소개한다. 각 대표 TC는 서로 다른 테스트 기법을 보여주도록 골랐다. 아래 표에서는 ★로 표시했다.

기능	대표 TC	무엇을 보여주나	기법
매치 시작	★ TC-A03 닉네임 10자 허용	입력 경계값을 1·2·10·11자로 나눠 검증	경계값 분석
라운드 판정	★ TC-B05 합 같음, 둘 다 더블 → 무승부	조건 조합을 결정 테이블로 빠짐없이 도출	결정 테이블
매치 진행·종료	★ TC-C01 3승 즉시 종료, 던지기 비활성화	상태가 바뀌는 시점과 화면 반응을 함께 검증 (Selenium)	상태 전이
중복 던지기	★ TC-E02 같은 라운드 동시 요청	동시성 문제를 부하 상황에서 검증	동시성·성능
통계	★ TC-F13 진행 중 매치는 승률에서 제외	화면 값을 DB 직접 조회 값과 비교	데이터 비교
사용자 흐름	★ TC-G01 3승 전에 페이지를 닫은 매치	기대 결과를 정할 수 없는 지점을 찾아 명세 이슈로 전환	시나리오·탐색

테스트 케이스 45건 작성
이 중 대표 6건으로 테스트 기법을 하나씩 보여준다: 경계값 · 결정 테이블 · 상태 전이 · 동시성 · 데이터 비교 · 시나리오

LINK ↗ docs/02-test-cases.md

[03] 환경 구성·설치 #installation-test

```
Terminal — 클린 설치 후 상태 확인 (2026-10-27)

$ docker compose up -d --build
Container dice-duel-qa-db-1 Started
Container dice-duel-qa-db-1 Healthy
Container dice-duel-qa-app-1 Started
Container dice-duel-qa-app-1 Healthy
Container dice-duel-qa-lb-1 Started
Container dice-duel-qa-generator-1 Started
(elapsed: 19s)

$ docker compose ps -a
SERVICE     IMAGE                   STATUS                    PORTS
app         dice-duel-qa-app        Up 32 seconds (healthy)   8000/tcp
db          postgres:16             Up 36 seconds (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
generator   dice-duel-qa-generator  Exited (0) 23 seconds ago
lb          nginx:1.27-alpine       Up 28 seconds             0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

$ curl -s http://localhost:8080/health
{"status":"ok","instance":"a2c6aea7a6a4","test_mode":true}
```

Docker Compose 클린 설치 19초, 설치 후 확인 6항목 PASS, 재시작 후 데이터 유지

LINK ↗ docs/03-install.md

[04] Sanity test #sanity

```
Terminal — Sanity test (2026-10-27 13:42)

$ bash scripts/sanity.sh
Sanity 2026-10-27 13:42:31 target=http://localhost:8080
PASS  SN-01  db·app 컨테이너 healthy  [app=healthy db=healthy lb= ]
PASS  SN-02  /health 응답 ok  [{"status":"ok","instance":"a2c6aea7a6a4","test_mode":true}]
PASS  SN-03  게임 화면: 규칙 '닉네임' 시작 '던지기' 요소  [4/4]
PASS  SN-04  매치 시작 (유효한 닉네임)  [match id=199]
PASS  SN-05  던지기 1회 → 라운드 1 기록  [rounds_played=1]
PASS  SN-06  매치 조회 API → DB  [api=1:0:1 db=1:0:1]
PASS  SN-07  통계 화면 200, 매치 목록 있음  [stats=200 total=199]
PASS  SN-08  합계 분포 2개 11분·3개 16행  [2개=11 3개=16]
PASS  SN-09  닉네임별 전적 응답 (봇 10명 이상)  [players=11]
PASS  SN-10  처리 log에 방금 라운드가 찍힘  [round processed id=199]
RESULT pass=10 fail=0

# 브라우저 수동 확인 (스크립트 범위 밖)
PASS  SN-11  게임 화면(브라우저) 시작·던지기  [started=true rolling true=false round=1 result=컴퓨터 승리 die1=6]
TOTAL pass=11 fail=0
```

11/11 PASS로 기능 검증 진행을 판단. 같은 스크립트를 Jenkins·Actions 첫 단계에서 재사용

LINK ↗ scripts/sanity.sh

---

reproduces: ← Part 1 · TC-002 테스트 설계와 수행 · Step 01~04

actual_result: 명세 확정 → TC 45건 · 유저 시나리오 5개 설계 → 클린 설치 · Sanity 통과 후 기능 검증 시작

status: PASS

sunghwan@qa: ~/dice-duel-qa

```
sunghwan@qa:~/dice-duel-qa$ ./run_test.sh --case TC-002 --target dice-duel-v1.0 --steps 05-06
```

PART 2 · TC-002 · STEP 05–06

기능 검증과 결함 리포트

명세 기반으로 검증하고, 결함은 영향까지 입증해 리포트한다
찾은 결함은 회귀 시나리오로 남긴다

RESULT
6 ISSUES
BUG 4 · SPEC 2

```
$ diff part1/TC-002 part2/dice-duel # 실무와 다른 점
```

PART 1 · 실무		PART 2 · DICE DUEL (Sample project)
패치 아이템별 기능 검증	→	v1.0 전체 기능 TC 45건 수행 (첫 릴리스로 가정) LINK ↗
Jira 결함 리포트	→	Markdown 결함 리포트 BUG-001~004 · 명세 이슈 SPEC-001~002 LINK ↗
BDD 시나리오 업데이트	→	회귀 시나리오 10건 등록, @R-xx · @BUG-xxx 태그로 feature 파일과 연결 LINK ↗

```
$ ls docs/05-defects/ # 리포트한 이슈 6건
```

ID	심각도	제목	출처	
[----------] **BUG** · 결함 4건				
BUG-001	Major	합과 더블이 모두 같아도 플레이어 승리로 판정	미리 심음	LINK ↗
BUG-002	Major	10자 닉네임이 거부됨 (화면 안내는 2~10자)	미리 심음	LINK ↗
BUG-003	Major	승률에 진행 중 매치가 포함되어 순위가 뒤바뀜	미리 심음	LINK ↗
BUG-004	Minor	매치 중에도 주사위 개수 선택이 바뀌어 보임	실제 발견	LINK ↗
[----------] **SPEC** · 명세 이슈 2건 · 명세에 규칙이 없어 기대 결과를 정할 수 없음				
SPEC-001	—	도중에 떠난 매치의 처리 규칙 없음	실제 발견	LINK ↗
SPEC-002	—	기간 필터의 시작일 > 종료일 처리 규칙 없음	실제 발견	LINK ↗

* 미리 심음: 결함 리포트를 보여주기 위해 일부러 넣은 결함 (위치와 의도를 planted-bugs.md에 공개)

[05] 기능 검증 결과 #manual-test



05. 기능 검증 결과 — Dice Duel v1.0

> Part 1 · TC-002 · 05 기능 검증·결함 리포트 수행: 2026-10-27 13:44~13:48 · 수행자: 김성환 · 빌드: main · 환경: 03-install.md · 선행: Sanity PASS 테스트 케이스: 02-test-cases.md · 원본 기록: evidence/

요약

전체	PASS	FAIL	BLOCKED	수행률	통과율 (수행 기준)
45	37	6	2	100%	86.0% (37/43)

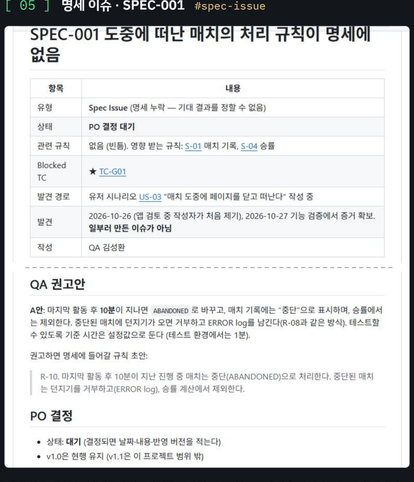
이슈	유형	심각도	제목	관련 TC
BUG-001	Bug	Major	합과 더블이 모두 같으면 무승부여야 하는데 플레이어 승리	★B05
BUG-002	Bug	Major	10자 닉네임이 거부됨 (화면 안내는 "2~10자")	★A03, A04
BUG-003	Bug	Major	승률에 진행 중 매치가 포함되어 순위가 뒤바뀜	★F13, F15
BUG-004	Bug	Minor	매치 중에도 주사위 개수 선택이 바뀌어 보임	A10
SPEC-001	Spec Issue	—	도중에 떠난 매치의 처리 규칙이 없음	★G01
SPEC-002	Spec Issue	—	기간 필터의 시작일 > 종료일 처리 규칙이 없음	F04

> BUG-004, SPEC-001, SPEC-002는 테스트 케이스와 유저 시나리오를 쓰면서 **실제로 발견한 것**이다. BUG-001~003은 결함 리포트를 보여주기 위해 미리 심은 것이며 planted-bugs.md에 기록되어 있다.

45건 수행: PASS 37 · FAIL 6 · BLOCKED 2. FAIL·BLOCKED는 모두 결함·명세 이슈 리포트로 연결

LINK ↗ docs/05-test-results.md

[05] 명세 이슈 · SPEC-001 #spec-issue



SPEC-001 도중에 떠난 매치의 처리 규칙이 명세에 없음

항목	내용
유형	**Spec Issue** (명세 누락 — 기대 결과를 정할 수 없음)
상태	**PO 결정 대기**
관련 규칙	없음 (빈틈). 영향 받는 규칙: S-01 매치 기록, S-04 승률
Blocked TC	★ TC-G01
발견 경로	유저 시나리오 US-03 "매치 도중에 페이지를 닫고 떠난다" 작성 중
발견	2026-10-26 (앱 검토 중 작성자가 처음 제기), 2026-10-27 기능 검증에서 증거 확보. **일부러 만든 이슈가 아님**
작성	QA 김성환

QA 권고안

A안: 마지막 활동 후 **10분**이 지나면 ABANDONED 로 바꾸고, 매치 기록에는 "중단"으로 표시하며, 승률에서는 제외한다. 중단된 매치에 던지기가 오면 거부하고 ERROR log를 남긴다(R-08과 같은 방식). 테스트할 수 있도록 기준 시간은 설정값으로 둔다 (테스트 환경에서는 1분).

권고하면 명세에 들어갈 규칙 초안:

> R-10. 마지막 활동 후 10분이 지난 진행 중 매치는 중단(ABANDONED)으로 처리한다. 중단된 매치는 던지기를 거부하고(ERROR log), 승률 계산에서 제외한다.

PO 결정

- 상태: **대기** (결정되면 날짜·내용·반영 버전을 적는다)
- v1.0은 현행 유지 (v1.1은 이 프로젝트 범위 밖)

명세에 규칙이 없어 기대 결과를 정할 수 없는 지점을 근거·QA 권고안과 함께 리포트. 결정은 결정권자와 협의할 아이템으로 남김

LINK ↗ docs/05-defects/SPEC-001.md

[05] 결함 리포트 · BUG-001 #defect-report KEY



BUG-001 합과 더블이 모두 같으면 무승부여야 하는데 플레이어 승리로 판정됨

항목	내용
유형 / 심각도 / 우선순위	Bug / **Major** / P1
상태	Open (v1.0 릴리스 전 수정 필요)
관련 규칙	R-02 합이 같으면 모든 눈이 같은 쪽이 승리. **둘 다 같으면 무승부**
관련 TC	★ TC-B05 (결정 테이블 8칸 중 이 칸만 FAIL)
발견	2026-10-27, 김성환, 기능 검증
환경	Dice Duel v1.0 (`main`), Docker Compose, Chrome — 03-install.md
발생 빈도	주사위 2개 매치에서 양쪽이 같은 더블이 나올 때마다 **항상**. 실제 데이터 3,358라운드 중 14회 (0.42%, 이론 1/216 = 0.46%)

기대 결과

- 라운드 결과 "**무승부**", 스코어 **0 : 0** (R-02: 합 6 = 6, 양쪽 모두 더블 → 무승부)

실제 결과

- 라운드 결과 "**플레이어 승리**", 스코어 **1 : 0**

실제 데이터에서 같은 조건 14회가 **모두** 플레이어 승리로 저장됨 (evidence/05-bug-001-occurrence.txt):

테스트 API 없이 실제 데이터를 조회해 3,358라운드 중 14회가 모두 잘못 저장된 것으로 영향을 입증

LINK ↗ docs/05-defects/BUG-001.md

[06] 회귀 시나리오 반영 #regression #bdd



06. 회귀 시나리오 — Dice Duel v1.0

> Part 1 · TC-002 · 06 회귀 시나리오 반영 작성: 2026-10-27 · 근거: 05-test-results.md의 FAIL·BLOCKED와 핵심 기능

결함은 **고쳐졌는지**만이 아니라 **다시 생기지 않는지**를 계속 확인해야 한다. 기능 검증에서 나온 결함과 핵심 기능을 회귀 시나리오로 등록하고, 가능한 것은 Gherkin으로 자동화해 매 빌드마다 돌린다.

언제 무엇을 돌리나

시점	범위	실행
매 빌드 (CI)	Sanity → `@smoke` 태그 시나리오	GitHub Actions · Jenkins
결함 수정 빌드	해당 결함의 RG + 같은 기능의 RG	CI + 수정 확인 수동
릴리스 전	전체 RG + 기능 TC 45건	CI + 수동

회귀 시나리오 목록

RG	출처	내용	원래 TC	우선순위	자동화	Feature 파일 · 태그
RG-01	BUG-001	라운드 판정 결정 테이블 **8칸 전체** (한 칸을 고치며 다른 칸이 깨지지 않	B01~B08	높음	UI (Selenium)	`02_round_judgement.feature` `@R-01 @R-02 @BUG-001`

결함·핵심 기능 10건을 회귀 시나리오로 등록하고, @R-xx · @BUG-xxx 태그로 feature 파일과 연결

LINK ↗ docs/06-regression.md

---

reproduces: ← Part 1 · TC-002 테스트 설계와 수행 · Step 05~06

actual_result: 결함 4건 · 명세 이슈 2건을 리포트하고, 회귀 시나리오 10건으로 다음 빌드부터 반복 검증

status: PASS

sunghwan@qa: ~/portfolio

```
sunghwan@qa:~/portfolio$ ./run_test.sh --case TC-003
```

TEST CASE · TC-003

테스트 자동화

Regression test · Web UI + Server
화면은 Selenium으로, 서버 동작은 Windmill로 검증한다

RESULT

6/6 PASSED

[######] 100%

```
$ cat tc-003.yaml
```

company: 비스텔 · 시높시스코리아

objective: 빌드·패치마다 반복되는 회귀 테스트를 Web UI와 서버 동작 양쪽에서 자동화

precondition:
- Java 기반 Web UI
- Airflow 기반 서버 job
- 빌드·패치마다 반복되는 회귀 테스트

environment: [Selenium, Cucumber (Gherkin), Jenkins, Windmill, Python, Airflow API, AI 코딩 도구, Jira]

labels: regression-testing · test-automation · ui-automation · backend-automation · bdd · ci-jenkins · job-monitoring · ai-assisted-testing

[==========] Running 6 steps from 2 suites

[----------] **Web UI** · Selenium + Cucumber + Jenkins · 4 steps

[PASSED] 01 **시나리오를 Gherkin으로 작성한다** #bdd #gherkin — KEY
└── Given / When / Then 형식으로 개발자·기획자도 읽을 수 있는 화면 시나리오를 만든다

[PASSED] 02 **Selenium으로 Step을 구현한다** #selenium #ai-assisted
└── Step definition을 Selenium으로 구현하고, AI 코딩 도구로 작성·개선한다 (결과는 직접 검증)

[PASSED] 03 **Jenkins로 실행한다** #jenkins #ci
└── Gherkin 시나리오를 Jenkins에 올려 빌드·패치마다 반복 수행한다

[PASSED] 04 **패치마다 시나리오를 유지보수한다** #maintenance
└── 패치 아이템으로 바뀐 동작은 기존 시나리오를 업데이트한다 ← TC-002

[----------] **Server** · Airflow API + Windmill (Python) · 2 steps

[PASSED] 05 **Server의 log를 수집한다** #airflow #api
└── job 성공·실패 여부, 처리된 데이터 결과, 소요 시간, log 정보를 Airflow API로 받아온다

[PASSED] 06 **필터링·모니터링을 스크립팅한다** #windmill #python #ai-assisted — KEY
└── 수집된 log 정보를 주기적으로 실행하는 자동화 스크립트로 필터링하여 이상을 모니터링한다

[==========] 6 steps from 2 suites ran.

```
$ cat features/02_round_judgement.feature                    # Part 2 · Dice Duel 실제 시나리오 (TC-B03)

Feature: 라운드 판정 - 결정 테이블 (R-01, R-02)
    Scenario: 합이 같으면 더블인 쪽이 라운드를 이긴다
        Given 사용자가 "후아" 닉네임으로 주사위 2개 매치를 시작했다
        And 다음 라운드 결과가 플레이어 [3, 3], 컴퓨터 [2, 4]로 정해져 있다
        When 사용자가 던지기 버튼을 누른다
        Then 라운드 결과는 "플레이어 승리"로 표시된다
        And 스코어는 1 : 0 이 된다
```

actual_result: Web UI 회귀 시나리오는 매 Patch 마다 Jenkins로 반복 수행하고, 서버 job은 주기적으로 모니터링

next: → Part 2 · Dice Duel · Cucumber 19 시나리오 CI 실행, log ERROR 17/17 검출

status: **PASS**

```
sunghwan@qa:~/dice-duel-qa$ ./run_test.sh --case TC-003 --target dice-duel-v1.0
```

PART 2 · TC-003 · STEP 01~06

테스트 자동화

화면은 Cucumber + Selenium을 CI로 반복 실행하고
서버는 log를 모아 ERROR를 걸러낸다

RESULT

0 NEW FAILS

PASS 15 · KNOWN FAIL 4

```
$ diff part1/TC-003 part2/dice-duel # 실무와 다른 점
```

PART 1 · 실무		PART 2 · DICE DUEL (Sample project)
Jenkins	→	Jenkins (Docker) + GitHub Actions (공개 실행 결과) LINK ↗
Airflow API로 job log 수집	→	인스턴스별 log 파일 수집 (Python) LINK ↗
Windmill 주기 실행 스크립트	→	Python log 모니터 --interval 주기 실행 LINK ↗
패치마다 시나리오 유지보수	→	범위 밖 · v1.1 패치 없음. 대신 @BUG 태그로 결함 수정 시 회귀 대상이 드러나게 함

[----------] Web UI · Cucumber + Selenium · Jenkins · GitHub Actions

[01] Gherkin 시나리오 작성 #gherkin

KEY

```
@R-01 @R-02
Feature: 라운드 판정 - 결합 테이블 (R-01, R-02)
  합이 큰 쪽이 이긴다. 합이 같으면 모든 눈이 같은 쪽(2개는 더블, 3개는 트리플)이 이기고,
  둘 다 같거나 둘 다 아니면 무승부다. 확정 눈은 테스트 모드 API로 지정한다 (R-07).
  회귀 시나리오: R6-01 (결합 테이블 8칸 전체, docs/06-regression.md)

  Scenario Outline: <TC> 플레이어 <플레이어> vs 컴퓨터 <컴퓨터> → <결과>
    Given 사용자가 "출target" 닉네임으로 주사위 개수는 개수는 개수를 시작한다
    And 다음 라운드 결과가 플레이어 <플레이어>, 컴퓨터 <컴퓨터>로 정해져 있다
    When 사용자가 던지기 버튼을 누른다
    Then 라운드 결과는 "<결과>"로 표시된다
    And 스코어는 <스코어> 이 된다

    Examples: 합이 다르다
      | TC     | 개수 | 플레이어 | 컴퓨터 | 결과       | 스코어 |
      | TC-R01 | 2    | [4, 5]   | [1, 2] | 플레이어 승리 | 1 : 0  |
      | TC-R02 | 2    | [1, 2]   | [6, 5] | 컴퓨터 승리   | 0 : 1  |

    Examples: 합이 같다 - 한쪽만 더블(이기나) 둘 다 아님
      | TC     | 개수 | 플레이어 | 컴퓨터 | 결과       | 스코어 |
      | TC-R03 | 2    | [3, 3]   | [2, 4] | 플레이어 승리 | 1 : 0  |
      | TC-R04 | 2    | [2, 4]   | [3, 3] | 컴퓨터 승리   | 0 : 1  |
      | TC-R05 | 2    | [1, 4]   | [2, 3] | 무승부       | 0 : 0  |

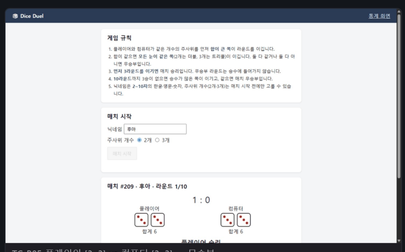
    @BUG-001
    Examples: 합이 같다 - 둘 다 더블
      | TC     | 개수 | 플레이어 | 컴퓨터 | 결과   | 스코어 |
      | TC-R05 | 2    | [3, 3]   | [3, 3] | 무승부 | 0 : 0  |

    Examples: 주사위 3개 - 2개만 같은 눈은 트리플이 아니다
      | TC     | 개수 | 플레이어 | 컴퓨터 | 결과   | 스코어 |
      | TC-R07 | 3    | [4, 4, 1] | [3, 3, 3] | 컴퓨터 승리 | 0 : 1 |
      | TC-R08 | 3    | [4, 4, 1] | [2, 2, 5] | 무승부 | 0 : 0 |
```

라운드 판정을 Scenario Outline 결정 테이블로 작성. 규칙 ID·결함 ID를 태그로 붙여 추적 (feature 4개, 시나리오 19개)

LINK ↗ automation/src/test/resources/features/

[02] Selenium으로 Step 구현 #selenium #page-object



TC-B05 플레이어 [3, 3] vs 컴퓨터 [3, 3] → 무승부

FAIL Then 라운드 결과는 "무승부"로 표시된다

data-testid selector와 Page Object로 구현. Step마다 스크린샷을 남겨 실패 화면(BUG-001)이 리포트에 첨부된다

LINK ↗ automation/src/test/java/qa/diceduel/

[03] Jenkins로 실행 #jenkins #ci

```
Jenkins (Docker) · Console Output — dice-duel-qa #3

Jenkins · dice-duel-qa #3 (Sanity → Smoke → Regression → Gate) — 결과 UNSTABLE: 알려진 결함 4건 FAIL, 새 실패 0

$ stage › Sanity
SKIP  SN-01  db-app 컨테이너 healthy  [docker 없음]
PASS  SN-02  /health 응답 200  [{"status":"ok","instance":"9646266fbe80f","test_mode":true}]
PASS  SN-03  게임 화면 : 규칙·'닉네임'·'시작'·'던지기' 요소  [4/4]
PASS  SN-04  매치 시작 (유효한 닉네임)  [match id=201]
PASS  SN-05  던지기 1회 → 주사위 3개, 합계, [rounds_played=1]
SKIP  SN-06  매치 조회 API → DB  [docker 없음]
PASS  SN-07  통계 화면 200, 매치 목록 있음  [stats=200 total=205]
PASS  SN-08  평균 분포 : 1개 3개 3개 3개  [2개13 3개16]
PASS  SN-09  닉네임별 전적 초입 (≥ 10판 이상)  [players=12]
SKIP  SN-10  처리 1회에 방금 라운드가 반영  [docker 없음]
RESULT pass=7 fail=0 skip=3

$ stage › Smoke (@smoke)
Scenario: 처음 온 사용자가 규칙을 보고 한 판을 시작한다 (US-01)
Scenario: 3승을 하면 매치가 즉시 끝나고 더 던질 수 없다 (TC-C01)

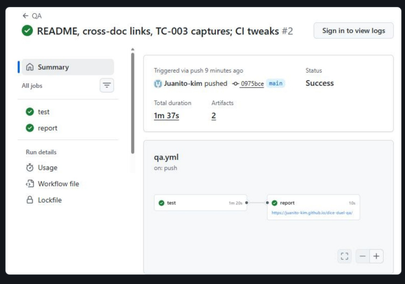
$ stage › Regression (전체)
(19 시나리오 실행 — 결과는 아래 Gate 참고)
Tests run: 19, Failures: 4 (Maven은 빌드만 보고 Gate가 판정)

$ stage › Gate (알려진 결함 외 실패 없음)
### Cucumber 최근 결과: 전체 19 · PASS 15 · 알려진 결함 FAIL 4 · 새 실패 0
| 알려진 결함 | @BUG-002 | 닉네임 길이 10자는 시작된다 |
| 알려진 결함 | @BUG-004 | 매치 중에는 주사위 개수를 바꿀 수 없다 (TC-A10) |
| 알려진 결함 | @BUG-001 | TC-B05 플레이어 [3, 3] vs 컴퓨터 [3, 3] → 무승부 |
| 알려진 결함 | @BUG-003 | 닉네임별 승률은 이긴 매치 ÷ 끝난 매치다 (TC-F12, F13) |
[OK] 알려진 결함 외 실패 없음
Finished: UNSTABLE
```

Sanity → Smoke → Regression → Gate 순서로 실행. 알려진 결함(@BUG) 외 실패만 빌드를 막도록 판정 기준을 설계 → 새 실패 0, 알려진 결함 4건으로 UNSTABLE

LINK ↗ jenkins/Jenkinsfile

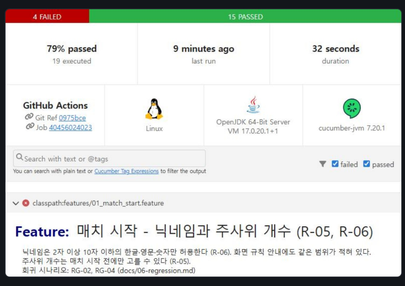
[03] GitHub Actions로 실행 #github-actions



push마다 같은 테스트를 실행해 공개 결과를 남긴다 (Success)

LINK ↗ .github/workflows/qa.yml

[03] 결과 리포트 공개 #report



Cucumber HTML 리포트를 GitHub Pages로 공개. 저장소를 받아 실행하지 않아도 결과를 볼 수 있다

LINK ↗ juanito-kim.github.io/dice-duel-qa/

[----------] Server · Python log monitor

[05] Server의 log를 수집한다 #log

```
Terminal — log 수집 (TC-003 · 05)

$ ls logs/                                  # 인스턴스별 log 파일
app-3099658be302.log
app-369569f115d8.log
app-376de14a284f.log
app-3771e9dff10e.log

$ head -n 12 monitoring/collected/2026-10-27.log   # 수집되어 하나로 모인 log
2026-10-27 14:20:00 ERROR [376de14a284f] match rejected: invalid nickname=' 유노 '
2026-10-27 14:20:00 ERROR [376de14a284f] match rejected: invalid nickname='유노!'
2026-10-27 14:20:00 ERROR [376de14a284f] throw rejected: match already finished id=221 round_no=5
2026-10-27 14:20:00 INFO [376de14a284f] match created id=215 nickname=베누스 dice=2 seed=175836149
2026-10-27 14:20:00 INFO [376de14a284f] match created id=216 nickname=디아나 dice=3 seed=532857417
2026-10-27 14:20:00 INFO [376de14a284f] match created id=217 nickname=유피테르 dice=2 seed=987187168
2026-10-27 14:20:00 INFO [376de14a284f] match created id=218 nickname=플루토 dice=2 seed=2018727246
2026-10-27 14:20:00 INFO [376de14a284f] match created id=219 nickname=유노 dice=2 seed=666035570
2026-10-27 14:20:00 INFO [376de14a284f] match created id=220 nickname=머큐로 dice=2 seed=1567774386
2026-10-27 14:20:00 INFO [376de14a284f] match created id=221 nickname=넵투누스 dice=2 seed=1686447431
2026-10-27 14:20:00 INFO [376de14a284f] match created id=222 nickname=미네르바 dice=3 seed=920296646
2026-10-27 14:20:00 INFO [376de14a284f] match created id=223 nickname=넵투누스 dice=2 seed=1401830183
```

인스턴스마다 따로 남는 log 파일을 모아 하나로 합친다

LINK ↗ monitoring/log_monitor.py

[06] ERROR 필터링 · 모니터링 #python #monitoring

KEY

```
Terminal — ERROR 필터링·경보 (TC-003 · 06)

$ docker compose run --rm monitor --once
[2026-10-27 14:20:03] 수집 319줄 → 인스턴스 3개 · INFO 299 · WARNING 5 · ERROR 12
  8  ERROR - 잘못된 닉네임 (R-06)
       ① 2026-10-27 14:20:00 ERROR [376de14a284f] match rejected: invalid nickname=' 유노 '
       ② 2026-10-27 14:20:00 ERROR [376de14a284f] match rejected: invalid nickname='유노!'
  5  WARNING - 중복 라운드 요청 (R-09)
       ① 2026-10-27 14:20:01 WARNING [376de14a284f] throw rejected: duplicate round id=237 round_no=1 expected=2
       ② 2026-10-27 14:20:01 WARNING [376de14a284f] throw rejected: duplicate round id=245 round_no=1 expected=2
  4  ERROR - 끝난 매치에 던지기 (R-08)
       ① 2026-10-27 14:20:00 ERROR [376de14a284f] throw rejected: match already finished id=221 round_no=5
       ② 2026-10-27 14:20:01 ERROR [376de14a284f] throw rejected: match already finished id=238 round_no=6
ALERT  ERROR 12건이 기준 10건을 넘음
```

규칙 ID별로 ERROR를 분류하고 기준을 넘으면 경보. 주입한 오류 17건 = 검출 17건으로 필터의 정확성을 검증

LINK ↗ monitoring/README.md

---

reproduces: ← Part 1 · TC-003 테스트 자동화 · Step 01~06

actual_result: Cucumber 19 시나리오를 Jenkins · GitHub Actions로 반복 실행하고, 서버 ERROR를 빠짐없이 검출

status: PASS

```
sunghwan@qa:~/portfolio$ ./run_test.sh --case TC-004
```

TEST CASE · TC-004

성능 테스트

Stress test · k8s pod scaling
사양 결정에 필요한 근거를 측정으로 만든다

RESULT

6/6 PASSED

[######] 100%

```
$ cat tc-004.yaml
```

company: 시높시스코리아

objective: 고객 환경별 최적 HW 사양 결정에 필요한 서버 스펙 측정 데이터 확보

precondition:
- Ubuntu 기반 Kubernetes (helm chart) 환경
- Kafka, Zookeeper, Spark, Airflow, PostgreSQL, Redis 등 서비스 pod

environment: [Kubernetes (helm chart), Kafka Publisher, Grafana, Prometheus, Windmill, Jira]

labels: performance-testing · stress-test · load-generation · k8s-pod-scaling · resource-monitoring

[==========] Running 6 steps from TC-004

[PASSED] 01 **측정 조건을 정한다** #test-plan
└ 고객 환경과 부하 조건을 정의한다

[PASSED] 02 **부하를 생성한다** #kafka #load-generation
└ Publisher로 Kafka에 대량의 메시지를 주입한다

[PASSED] 03 **모니터링을 구성한다** #grafana #prometheus
└ Grafana, Prometheus로 서비스·인프라 상태를 모니터링한다

[PASSED] 04 **인스턴스 수를 바꿔가며 측정한다** #k8s-pod-scaling KEY
└ 서비스별 인스턴스 수를 바꿔가며 인스턴스당 CPU·메모리 점유 등 서버 스펙을 측정한다

[PASSED] 05 **반복 측정을 자동화한다** #windmill
└ 같은 조건으로 반복 측정할 수 있도록 테스트 스크립트 실행을 자동화한다

[PASSED] 06 **측정 결과를 제출한다** #report
└ 측정 결과를 정리해 아키텍처팀에 제출하고, 최적 사양 결정의 근거로 제공한다

[==========] 6 steps ran.

---

actual_result: 측정 결과가 아키텍처팀의 고객 환경별 최적 HW 사양 결정 근거로 활용됨

next: → Part 2 · Dice Duel · 인스턴스 1→3개, 처리량 188→534 요청/초 (×2.84)

status: PASS

sunghwan@qa: ~/dice-duel-qa

```
sunghwan@qa:~/dice-duel-qa$ ./run_test.sh --case TC-004 --target dice-duel-v1.0
```

PART 2 · TC-004 · STEP 01–06

성능 테스트

같은 부하에서 인스턴스 수를 1 → 2 → 3개로 바꿔가며
처리량, 응답 시간, 자원 사용을 측정한다

RESULT

×2.84 THROUGHPUT

188 → 534 req/s · 오류 0

```
$ diff part1/TC-004 part2/dice-duel # 실무와 다른 점
```

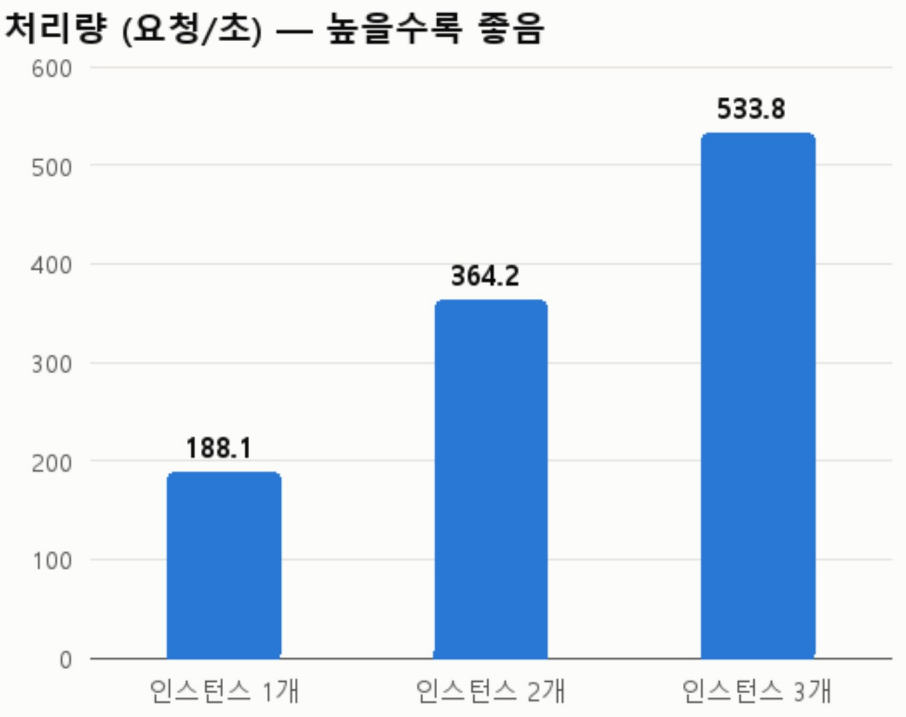
PART 1 · 실무		PART 2 · DICE DUEL (Sample project)
고객 환경·부하 조건 정의	→	개발 PC 1대 (CPU 16 · 16GB) · nginx → app × N → PostgreSQL 16
Kafka Publisher로 메시지 주입	→	가상 사용자(봇, Generator) 매치 1,000개 · 동시 20개 (약 5,500 요청)
Grafana · Prometheus 모니터링	→	docker stats 2초 간격 (CPU · 메모리), DB 직접 조회
k8s pod 수 조정	→	docker compose --scale app=1 · 2 · 3
Windmill 반복 측정 자동화	→	perf/run_perf.sh 한 번으로 같은 조건 반복 측정 LINK ↗
아키텍처팀에 결과 제출	→	perf/report.md 결과 리포트 LINK ↗

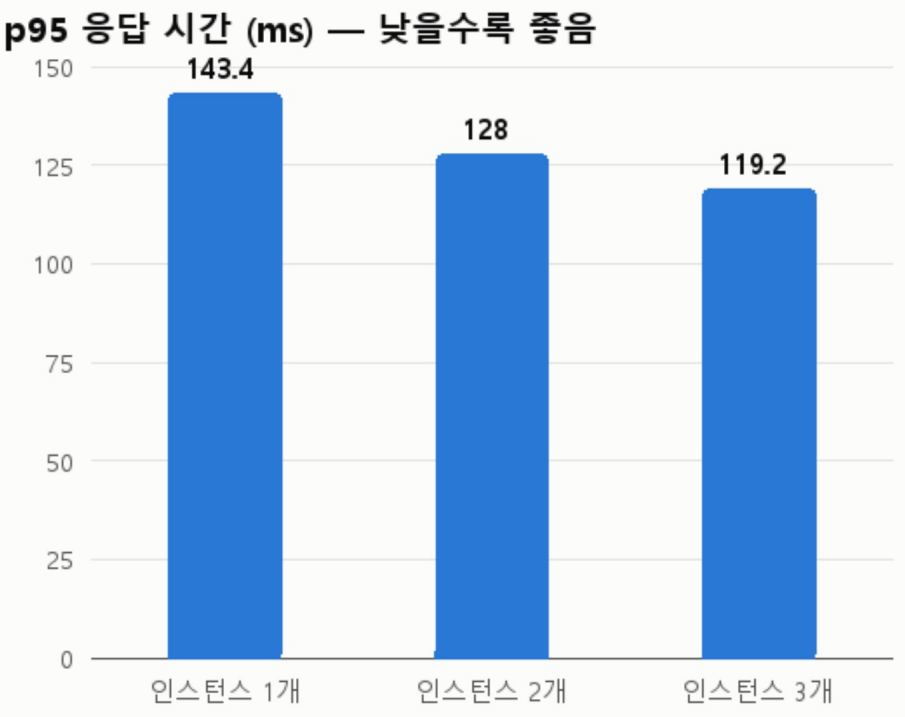
[04] 인스턴스 수를 바꿔가며 측정한다 #scaling

KEY

인스턴스 수별 성능 (봇 매치 1,000개 · 동시 20개, 약 5,500 요청)

같은 부하에서 app 인스턴스를 1·2·3개로 늘렸을 때 · 오류 0건 · 부하 중 같은 라운드 동시 요청 100쌍 모두 1건만 처리





처리량은 인스턴스 수에 거의 비례해 늘었다 (×1.94, ×2.84). 평균 응답은 106 → 37ms로 줄었지만 p95는 143 → 119ms로 덜 줄었다

LINK ↗ perf/report.md

[06] 측정 결과를 정리한다 #report

처리량과 응답 시간

인스턴스	처리량 (요청/초)	평균 응답	p95 응답	최대 응답	오류
1	**188**	105.9ms	**143.4ms**	201.7ms	0
2	**364** (×1.94)	54.6ms	**128.0ms**	158.6ms	0
3	**534** (×2.84)	37.2ms	**119.2ms**	169.7ms	0

부하 분배와 자원 사용

인스턴스	요청 수 · 소요 시간	인스턴스별 처리 건수	app CPU 최고 (합)	DB CPU 최고
1	5,552건 · 29.5초	5,552	169%	20%
2	5,547건 · 15.2초	2,761 / 2,786	238%	37%
3	5,463건 · 10.2초	1,816 / 1,817 / 1,830	309%	53%

CPU 100% = 코어 1개. app 메모리는 인스턴스당 43~57MB로 부하와 상관없이 거의 일정했다.

요청이 인스턴스별로 고르게 분배됐다. app 메모리는 인스턴스당 43~57MB로 부하와 상관없이 거의 일정

LINK ↗ perf/report.md

```
$ cat perf/report.md | findings
```

- 부하 중 같은 라운드 동시 요청 300쌍이 모두 1건만 처리되고 DB 중복 0 (다른 인스턴스로 갈라진 쌍 최대 99)
- DB CPU가 20% → 53%로 함께 올라감. 인스턴스 4개 이상에서는 DB가 다음 병목 후보
- 첫 측정에서 부하 발생기 실행이 app을 1개로 되돌린 것을 instance_counts로 알아채고 고쳐서 재측정
- 한계: 모든 구성 요소가 한 PC에서 실행된 측정

---

reproduces: ← Part 1 · TC-004 성능 테스트 · Step 01~06

actual_result: 인스턴스 3개까지 오류 0, 처리량 ×2.84. 다음 병목은 DB로 판단해 리포트에 기록

status: PASS

sunghwan@qa: ~/portfolio

```
sunghwan@qa:~/portfolio$ ./show_milestones.sh --from 2012
```

ROADMAP · 2012 – NOW

경력 마일스톤




```
$ cat milestones.log
```

M1	[x]	2012.10	QA 커리어 시작 · Telematics, Infotainment 명세 기반 검증
M2	[x]	2018.04	도메인 전환 · Automotive → Semiconductor (Big data 기반 Smart factory)
M3	[x]	비스텔	테스트 자동화 · BDD 기반 Web UI 자동화 스크립트 작성
M4	[x]	시높시스	인프라 성능 테스트 · k8s(Helm chart) 환경에서 최적 HW 사양 결정을 위한 서버 스펙 측정
M5	[x]	시높시스	AI 활용 자동화 · AI 코딩 도구로 테스트 스크립트 작성·개선, 결과는 직접 검증
NEXT	[ ]	2026 –	[다음 마일스톤 · 함께할 팀]

sunghwan@qa:~/portfolio$ ls -R ~/tools

APPENDIX · A

Appendix

전체 도구 목록

test-automation/

Selenium
Cucumber (Gherkin)

ci-scheduling/

Jenkins
Windmill

test-management/

Jira
Zephyr (for Jira)
Confluence

performance/

Grafana
Prometheus
Kafka Publisher

infra-platform/

Kubernetes (helm)
Docker Compose
Kafka · Zookeeper
Spark · Airflow
PostgreSQL · Redis

os/

Linux (Ubuntu)
Windows

languages/

Python
Gherkin

ai/

GitHub Copilot

sunghwan@qa:~/portfolio$